SD2 — Distributed Rate Limiter (Amazon SDE II / Google L4 bar, L5 extensions)

> Oracle: recruiter-flagged; reported as both HLD ("rate limiter for an API gateway") and LLD ("implement token bucket class"). This file covers both; §10.1 is the LLD. Tags: `[hot path]` · `[async]` · `[control plane]`.
> ❓ = follow-up with the answer you give.

Index

---

§0 Time budget

Clarify 4 · why-hard 3 · FR/NFR 4 · BoE 2 · HLD 8 · API/data 4 · deep dives 12 · tradeoffs/wrap 4 → 41

🗣 "Two axes I'll settle first: where the limiter sits (gateway vs library) and how strict the count must be (exact vs approximate). Those two decide everything."

§1 Why this is hard

```
naive: in-memory counter per client in each API server
breaks at: 2 servers → client gets 2× limit; restart → counter resets to 0
```

- Naive: `Map<clientId, count>` reset every minute in each server
- First break: horizontal scaling. N servers = N× the limit. Then restart resets. Then the minute boundary lets 2× through (100 at 0:59, 100 at 1:00)
- Minimum primitive: **a shared counter with atomic increment** (Redis) — not consensus, not a DB. Redis `INCR` is single-threaded atomic; that's the whole trick
- User wants: "if my limit is 100/s, I get 100/s, not 40 because of bad luck; and when I'm over, tell me when to retry"
- Cost of being wrong → too lenient: backend overload (the thing the limiter protects) — availability cost → too strict: rejected legitimate traffic — revenue cost → so: approximate is fine, **fail-open on limiter failure**, and precision matters only near the limit

🗣 "This is a **shared-counter problem at ~100k checks/s**, because correctness only matters within a few percent and availability of the limiter must never be worse than the thing it protects."

❓ *Why not do this in the load balancer / cloud WAF?* → "For per-IP flood protection, yes, buy it (WAF rate rules). For per-tenant, per-API-key, per-endpoint quotas with different windows and dynamic limits, the LB doesn't know the tenant. This is an application-layer limiter behind the LB."

§2 Crux

- Hard part: **one atomic read-modify-write per request, shared across a fleet, at sub-millisecond cost** — and staying available when the shared store isn't
- Critical points → algorithm choice: token bucket for burst-tolerant limits; sliding window for "N per window" fairness → atomicity: Lua script or single command; never GET-then-SET → latency budget: limiter adds < 1 ms p99 or it becomes the bottleneck → fail-open with alarm → correct headers so clients back off instead of hammering
- Toy vs runnable: toy = one Redis key per client. Runnable = key TTLs, Redis cluster slotting, local cache tier, per-limit config reload without deploy, metrics on reject rate per tenant
- Business metric: "% of backend overload minutes" ↓ and "% legit requests rejected" ↓; system metric: `limiter_decision_latency_p99`, `rejects_per_tenant`, `limiter_unavailable_seconds`

🗣 "Crux: atomic shared counting that fails open, so I'll design around Redis Lua with a local pre-filter."

§3 Clarifying questions

Q	Assumed
Where does it sit?	API gateway middleware, all requests
Keyed by?	tenant (API key), optionally per endpoint
Scale?	100k req/s peak across fleet, 10k tenants
Limits?	per-tenant: 1k/s burst 2k; per-endpoint overrides; configurable at runtime
Accuracy?	±5% acceptable; hard cap not required
Behavior on limiter failure?	fail-open (allow) with alarm
Response on reject?	429 + `Retry-After` + `X-RateLimit-*` headers
Multi-region?	per-region limits (limits are regional); global optional later

§4 FRs

- Core: check-and-consume per request; configurable limit per (tenant, endpoint); 429 with headers; runtime config update
- Extended: quotas (daily), tiered plans, dry-run/shadow mode, admin override
- Non-goals: exact global counting across regions; billing metering (separate system)

§5 NFRs

Req	Target	Why	Enforced by
Decision latency	p99 < 1 ms added	on every request	local cache + Redis same-AZ, pipelined
Accuracy	±5% of limit	approximate OK	sliding-window counter / token bucket
Availability	limiter never lowers gateway availability		fail-open + timeouts 5 ms
Consistency	eventual across fleet; atomic per key		Redis single-threaded ops
Throughput	100k checks/s		Redis cluster ~100k+ ops/s per node order of magnitude (Redis docs benchmarks) → 3–6 nodes
Durability	none required (counters are ephemeral)		no persistence needed; AOF off
Cost	order of magnitude $500–1500/mo (assume)		3-node Redis + config svc
Security	tenant id from authenticated key only	spoofing	resolve tenant after authn
Degradation	Redis down → allow all + alarm; config svc down → last-known limits		

L5: decide *what you're protecting*: backend capacity (then a global "total" limiter matters more than per-tenant) vs fairness/monetization (per-tenant). Most real systems need both: per-tenant limit *and* a fleet-wide concurrency cap — that second one is a semaphore, not a rate limiter.

§6 BoE

- 100k req/s × 1 Redis op (Lua, 1 round trip) = 100k ops/s → 2–3 Redis nodes at ~50k ops/s conservative
- Keys: 10k tenants × ~5 endpoints = 50k keys × ~100 B = 5 MB — trivial; TTL = window
- Latency: same-AZ RTT ~0.2–0.5 ms order of magnitude + Lua exec µs → fits < 1 ms
- Local pre-filter: if a tenant is already > 2× limit locally, reject without Redis → cuts Redis load under attack
- So what: Redis is not the bottleneck; **network round trip per request is** — batch/pipeline or add local tier

§7 HLD

```
[hot path]  Client → LB → API GW pod ─┬→ authn (API key → tenant)
                                      ├→ Limiter middleware
                                      │     ├ local token cache (per pod, short TTL)   [hot path]
                                      │     └ Redis Cluster: EVALSHA sliding-window    [hot path]
                                      └→ upstream service
[control plane]  Limit config svc → pushes rules (tenant, endpoint, limit, window) → pods (watch/poll
10 s)
[async]          Metrics: allow/reject per tenant → Prometheus; rejects → alerting
```

- Components → Limiter middleware `[hot path]`: decides allow/deny, sets headers → Redis Cluster `[hot path]`: atomic counters; key = `rl:{tenant}:{endpoint}:{window}` → Local cache `[hot path]`: per-pod hot-tenant pre-filter and fail-open fallback → Config service `[control plane]`: source of truth for limits (Postgres), pushes to pods → Metrics `[async]`

Primary path — check request

1. GW authenticates → tenant `T`, endpoint `E` — fails → 401, never reaches limiter
2. Middleware looks up rule `(T,E)` in local config map (refreshed every 10 s) — missing → default rule
3. Local pre-filter: if pod-local counter for `(T,E)` > 3× limit in last second → 429 immediately (attack shedding), skip Redis — this is approximate by design
4. `EVALSHA sliding_window.lua 1 rl:{T}:{E} now_ms window_ms limit` with 5 ms timeout — returns `{allowed, remaining, retry_after_ms}`
5. Timeout/error → **allow**, increment `limiter_fail_open`, alarm if > 1% for 30 s
6. Allowed → set `X-RateLimit-Limit/Remaining/Reset`, forward upstream; denied → 429 + `Retry-After`
7. Async: emit decision metric

- Consistency boundary: the Redis key. Config is eventually consistent (≤10 s stale)
- Contention: hot tenant key on one Redis slot — one tenant at 50k/s saturates one node; see §10.3

❓ *Why check after authn, not before?* → "Rate limiting by unauthenticated IP is a different limiter (WAF). Tenant limits need the tenant. Do IP limits at the edge, tenant limits here."

L5: the local pre-filter changes semantics: under a flood, the pod rejects locally *before* Redis sees it, so Redis undercounts — acceptable because the goal under flood is shedding, not accounting. Say that explicitly; interviewers like knowing you know the count is wrong on purpose.

§8 API

- Data path: none new — middleware. Response headers (IETF draft `RateLimit-*`):

```
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 12
X-RateLimit-Reset: 1725000060      (epoch s)
Retry-After: 1                     (on 429)
```

- Control plane:

```
PUT  /v1/limits/{tenant}          If-Match: version
{ "default": {"rps":1000,"burst":2000}, "endpoints": {"POST /orders": {"rps":100,"burst":100}} }
GET  /v1/limits/{tenant}
POST /v1/limits/{tenant}:reset    (admin; clears counters)
```

- Internal: pods poll `GET /v1/limits?since=version` or subscribe to a Redis pub/sub channel `limits.changed`

§9 Data model

- Redis (ephemeral) → sliding-window counter: two keys per (tenant, endpoint): `rl:{T}:{E}:{curWindowStart}` and `…:{prevWindowStart}`, INCR + EXPIRE 2×window → token bucket: one hash `tb:{T}:{E}` = `{tokens, last_refill_ms}`, EXPIRE = time to refill full → hash tag `{T}` in key so all a tenant's keys land on one slot (Lua multi-key requires same slot)
- Postgres (config): `limits(tenant_id, endpoint, rps, burst, window_ms, version, updated_at)`
- Retention: none; PII: tenant id only

❓ *Why hash tag the tenant?* → "Redis Cluster runs a Lua script only if all keys hash to one slot. `{T}` forces that. Cost: a hot tenant is one slot — that's the tradeoff you accept for atomicity."

§10 Deep dives

10.1 Algorithm choice + LLD (the Oracle LLD version)

Algorithm	Buys you	Costs you	Choose when
Token bucket	bursts up to `burst`; smooth avg	tuning 2 params	API limits, default choice
Leaky bucket	perfectly smooth output	queueing latency; drops bursts	shaping egress to a fragile downstream
Fixed window	1 INCR, trivial	2× at boundary	internal coarse limits
Sliding window log	exact	O(N) memory per key	small N, exactness required (login attempts)
Sliding window counter	~exact, O(1)	±small error (Cloudflare's analysis: 0.003% error on their traffic)	high-volume "N per window"

- Pick: **token bucket** for tenant API limits (burst is a feature); **sliding window counter** for "N per minute" quotas

Token bucket, thread-safe, single process (LLD answer):

```python
import time
from threading import Lock

class TokenBucket:
    """Lazy-refill token bucket; thread-safe; no background timer."""
    def __init__(self, capacity, refill_per_sec):
        self.capacity = capacity
        self.rate = refill_per_sec
        self.tokens = float(capacity)
        self.last = time.monotonic()
        self.lock = Lock()

    def try_acquire(self, n=1):
        with self.lock:
            now = time.monotonic()
            self.tokens = min(self.capacity, self.tokens + (now - self.last) * self.rate)
            self.last = now
            if self.tokens >= n:
                self.tokens -= n
                return True
            return False
# Time: O(1); per-key: dict key -> TokenBucket, evict idle keys by last-access TTL
```

```java
public final class TokenBucket {
  private final long capacity, refillPerSec;
  private double tokens;
  private long lastNs;
  public TokenBucket(long capacity, long refillPerSec) {
    this.capacity = capacity; this.refillPerSec = refillPerSec;
    this.tokens = capacity; this.lastNs = System.nanoTime();
  }
  public synchronized boolean tryAcquire(int n) {
    long now = System.nanoTime();
    tokens = Math.min(capacity, tokens + (now - lastNs) / 1e9 * refillPerSec);
    lastNs = now;
    if (tokens >= n) { tokens -= n; return true; }
    return false;
  }
}
```

- Lazy refill: no background thread; refill computed on access → no timer, no drift
- `synchronized` is fine: critical section is ~50 ns; contention only matters > ~1M calls/s per bucket. Lock-free version: pack `(tokens, lastNs)` into an `AtomicLong` CAS loop — mention, don't write unless asked
- Per-key map: `ConcurrentHashMap<Key, TokenBucket>` with `computeIfAbsent`; eviction of idle keys via Caffeine `expireAfterAccess`

❓ *Make it lock-free* → "CAS on a packed long: read state, compute new, `compareAndSet`, retry on failure. Under contention CAS retries can exceed the cost of the lock; measure first." ❓ *Multiple buckets, e.g., per-second and per-day?* → "Chain them; acquire from all or none — check both, then consume both; there's a tiny window where check passes and consume fails on the second; accept or hold a lock across both."

Sliding-window counter in Redis (distributed answer):

```
-- KEYS[1]=base key  ARGV: now_ms, window_ms, limit
local now, win, limit = tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3])
local cur = math.floor(now / win) * win
local prev = cur - win
local kc, kp = KEYS[1]..':'..cur, KEYS[1]..':'..prev
local pc = tonumber(redis.call('GET', kp) or '0')
local cc = tonumber(redis.call('GET', kc) or '0')
local weight = 1 - (now - cur) / win
local est = pc * weight + cc
if est + 1 > limit then
  return {0, 0, math.ceil(cur + win - now)}
end
redis.call('INCR', kc); redis.call('PEXPIRE', kc, win * 2)
return {1, math.floor(limit - est - 1), 0}
```

Caller side (Python, redis-py) — load once, EVALSHA per request:

```
class SlidingWindowLimiter:
    def __init__(self, r, script_src, limit, window_ms):
        self.r, self.limit, self.window_ms = r, limit, window_ms
        self.sha = r.script_load(script_src)          # once at startup

    def allow(self, tenant, endpoint):
        key = f"rl:{{{tenant}}}:{endpoint}"           # {tenant} hash tag -> one slot
        now_ms = int(time.time() * 1000)
        ok, remaining, retry_ms = self.r.evalsha(
            self.sha, 1, key, now_ms, self.window_ms, self.limit)
        return bool(ok), remaining, retry_ms          # surface Retry-After
# Fail-open on redis.ConnectionError (count locally + alarm) — availability over precision
```

- Atomic: whole script runs on one thread; no GET-then-SET race
- Error: assumes uniform distribution in previous window; bounded, small

L5: the script must be short — Redis blocks all clients while a Lua script runs; a 1 ms script at 100k/s is 100% CPU. Keep it O(1), avoid loops. Also `EVALSHA` not `EVAL` (don't ship the script text per call).

10.2 Atomicity, race, and clock (data/consistency)

- Race: two pods GET count=99, both SET 100 → 101 allowed. Fix: script (above) or `INCR` first then check `> limit` and undo with `DECR` only if you need exactness (usually not)
- Clock: use Redis `TIME` inside the script, not pod clocks → all pods share one clock; pod skew irrelevant

- Multi-key atomicity across tenants (fleet-wide cap + tenant cap): two scripts, two round trips, non-atomic → accept small over-admission, or put both keys under one hash tag (then all tenants on one slot — no). Accept

Sequence (allow path with fail-open):

1. Pod sends `EVALSHA` with 5 ms deadline
2. Redis executes atomically, returns
3. Pod applies decision; on `NOSCRIPT` → `SCRIPT LOAD` once, retry — on timeout → allow, count `fail_open`
4. If `fail_open` rate > 1% for 30 s → alarm; if > 50% → pods switch to local-only token buckets sized at `limit / pod_count` until Redis recovers (degraded but bounded)

L5: fail-open is a policy, not a law — for a login endpoint, fail-closed (deny) is correct because the limiter is a security control there. Make failure mode per rule.

10.3 Hot key / hot tenant (flow control)

- Problem: one tenant at 50k/s → one Redis slot → one node saturated; other tenants on that node suffer
- Options → **local pre-filter** (§7 step 3): shed at pod before Redis → **key sharding**: `rl:{T}:{E}:{shard = pod_id % 8}` each with `limit/8` — accuracy drops (uneven pod traffic) but load spreads. Choose when one tenant > ~20k/s → **client-side batching**: pod grabs 100 tokens from Redis at once, serves locally (like a "token lease") — reduces Redis ops 100×, error bounded by batch size
- Pick: pre-filter always; token lease for top-10 tenants; key sharding as last resort

Sequence (token lease):

1. Pod's local bucket for `(T,E)` empty → `EVALSHA lease.lua` requesting min(100, remaining)
2. Redis decrements by granted amount atomically, returns granted
3. Pod serves next `granted` requests locally, no Redis calls
4. Pod dies with 60 unused tokens → tenant temporarily undercounted by 60 — acceptable; leases expire with the window

L5: this is exactly how cloud-provider gateways get per-tenant limits at millions of RPS — a two-tier "global allocator + local spender" design. Name it so the interviewer sees you've read about it (Google's "Rate limiting at scale" / Stripe's limiter post).

10.4 Config propagation + safety (control plane)

- Limits change at runtime (support raises a tenant's cap during an incident)
- Design: Postgres → config svc → pods poll every 10 s with ETag; optional pub/sub for < 1 s
- Safety: validate (rps ≤ hard max), version + `If-Match`, audit log who changed what; **shadow mode** flag per rule = compute decision, log it, don't enforce — used to roll out new limits without breaking customers
- Failure: config svc down → pods keep last-known rules (in-memory + disk snapshot); pod cold start with config down → default rules, alarm

L5: the killer bug is a fat-finger `rps=1` on the default rule that rejects all traffic in 10 s fleet-wide. Guardrails: change limits by ≤10× per edit without a break-glass flag; canary config to 1 pod first.

§11 Hygiene

Concern	Default	Bites here
AuthN/Z	tenant from API key	never limit on client-supplied header
Idempotency	n/a data path; config PUT with If-Match	
Timeouts/retries	Redis 5 ms, no retry (retry doubles load)	
Rate limiting	this is it; plus WAF IP limits at edge	
Circuit breaker	on Redis: open → local mode	
Queues	none; metrics async	
Caching	rules cached 10 s; token lease	staleness after limit change
Consistency	atomic per key; eventual config	
Storage/partition	Redis Cluster slots by `{T}`	hot tenant
Replication/failover	Redis replica per shard, async; failover loses ≤1 s of counts — fine	
Observability	decision latency, reject rate per tenant, fail_open, hot-key top-N	
Deploy/rollback	shadow mode; canary pod	
Retention/PII	none	
Multi-tenancy	key per tenant; fleet-wide cap separately	

§12 Ops readiness

- Capacity: Redis ops/s per node; headroom 50%; trigger: node CPU > 60% or p99 > 2 ms → add shard (resharding is online in Redis Cluster)
- Runbook → **429 spike for one tenant** → check rule change audit → hot-key dashboard → talk to tenant, not fleet → **fail_open rising** → Redis health → failover or scale; confirm degraded local mode engaged → **upstream overload despite limiter** → fleet cap missing or set too high → lower via config (canary first)
- Migration: change algorithm by dual-writing new keys in shadow mode; compare decisions; flip
- Fallbacks: Redis down → local buckets sized `limit/pods`; config down → last-known
- Testing: load 100k/s synthetic; chaos: kill Redis primary, verify failover < 2 s and fail_open counted; correctness test: 1,000 req at limit 100 in 1 s → 100±5 allowed

§13 Pop-ups

- *Why not Postgres counters?* → 100k RMW/s on one row is far past a single Postgres row's lock throughput (order of magnitude 5–10k/s); Redis does it in memory single-threaded
- *Why not in-memory per pod with sticky routing?* → Sticky by tenant makes pods hot and breaks on scale-in; acceptable for small fleets with consistent hashing at the LB — mention as a cheaper v0
- *Redis primary dies?* → Sentinel/cluster failover ~1–2 s; counters on replica may lag by ms; you over-admit briefly. Acceptable

- *Client at 999/s sees random 429s?* → Precision: token bucket with `burst ≥ rps` gives no rejects at exactly the limit; sliding counter can reject near boundary; headers tell them remaining
- *Distributed across regions?* → Regional limits; global quota reconciled async (daily quota counted per region, summed hourly). Synchronous global counting = cross-region RTT on every request; no
- *How do you test accuracy?* → Deterministic clock injection; property test: allowed ≤ limit + burst over any window
- *What if a tenant bypasses with many API keys?* → Limit per account above per key; abuse detection separate
- *10×: 1M/s?* → Token lease tier becomes mandatory; Redis ops drop 100×; then Redis is fine, pods' local CPU is the cost

§14 Trade-offs

Decision	Chosen	Alt	Flips
Store	Redis cluster	in-pod + sticky LB	fleet < 5 pods, coarse limits
Algorithm	token bucket (API), sliding counter (quota)	fixed window	internal only, boundary burst OK
Failure	fail-open	fail-closed	limiter is a security control
Precision	±5%	exact (window log)	small N, abuse/login
Placement	gateway middleware	library in each service	services need own limits on internal calls

- Defeaters → "Fixed window is fine" — boundary lets 2× through; say why → "Make it exact" — exactness costs a round trip per request and memory per event; nobody needs exact at 100k/s → "Put a queue in front" — a rate limiter rejects; a queue delays. Mixing them turns 429s into latency, which is worse for most APIs

§15 Wrap-up

"Crux: atomic shared counting that fails open. Design: gateway middleware, Redis Lua sliding-window/token-bucket keyed by tenant with hash tags, local pre-filter and token lease for hot tenants, config service with shadow mode. NFRs: < 1 ms added p99, ±5%, never reduces availability. MVP0: one Lua script, one Redis, headers, fail-open with alarm. Week-one: decision latency, reject rate per tenant, fail_open. At 10×: token lease everywhere, key sharding for the top tenants."

§16 Card

- Pitch: Redis Lua atomic counters keyed by tenant, local pre-filter, fail-open, token lease for hot keys
- Why hard: N servers × N limits, and GET-then-SET races
- Numbers: 100k/s · < 1 ms p99 · ±5% · 5 ms Redis timeout · 10 s config refresh
- Failures: hot tenant slot · Redis down (fail-open) · fat-finger config
- Defeaters: fixed window · exactness · queue instead of reject
- MVP0 Lua + headers → MVP1 config svc + shadow → MVP2 token lease + fleet cap
- Done when: write token bucket class and Lua script from memory; explain sliding-counter error in 20 s

§17 References

- github.com/bucket4j/bucket4j — Java token bucket, Redis backends (read the Lua)
- github.com/envoyproxy/ratelimit — production limiter design
- Cloudflare blog, "How we built rate limiting capable of scaling to millions of domains" (sliding window counter)
- Stripe blog, "Scaling your API with rate limiters"
- Redis docs: EVAL/EVALSHA scripting semantics, cluster hash tags